PredictX

Decentralized Prediction Market Protocol

Technical Whitepaper v1.0

March 2025 | Sepolia Testnet

https://prediction-market-d-app.vercel.app

Author : Aditya Chotaliya

Abstract

PredictX is an open-source, fully on-chain prediction market protocol deployed on the Ethereum Sepolia testnet. The protocol enables permissionless creation and participation in prediction markets using a parimutuel pooling model, where the total ETH pool from all traders is redistributed to winning participants proportionally.

Unlike order-book based prediction markets, PredictX uses a continuous parimutuel model where probabilities are dynamically derived from the ratio of ETH committed to each outcome. This design eliminates the need for market makers while maintaining economically rational pricing.

The protocol integrates Chainlink price feeds for real-world data, The Graph for efficient event indexing, and a multi-layered incentive system consisting of a native PRED token and an on-chain referral mechanism.

1. Introduction

1.1 Background

Prediction markets are among the most accurate forecasting tools known. Academic research consistently demonstrates that aggregated market prices outperform expert opinion in forecasting real-world outcomes (Sunstein, 2006; Arrow et al., 2008). Centralized prediction markets, however, suffer from counterparty risk, censorship, geographic restrictions, and opacity.

Decentralized prediction markets address these limitations by moving market logic entirely on-chain. PredictX builds upon this premise, providing a protocol that is:

- Permissionless — anyone can create a market on any topic
- Trustless — resolution via on-chain oracles, no human custodian
- Transparent — all market state is publicly verifiable on-chain
- Composable — integrates with DeFi primitives (ERC20 tokens, Chainlink)

1.2 Problem Statement

Existing on-chain prediction market solutions suffer from:

- High gas costs due to complex AMM or CLOB mechanisms
- Liquidity fragmentation across outcomes
- Limited oracle integrations for automatic resolution
- No incentive mechanism to bootstrap market creation

PredictX addresses each of these through a simplified parimutuel model with integrated PRED token incentives and an on-chain referral system.

1.3 Solution Overview

PredictX implements a two-tier market system:

- Binary Markets (YES/NO) — Simple directional prediction markets
- Multi-Outcome Markets — 2 to 10 possible outcomes per market

Both market types use the same parimutuel pooling mechanism, with resolution triggered by an oracle contract after market expiry.

2. Protocol Design

2.1 Parimutuel Model

In the PredictX parimutuel model, all ETH deposited by traders (minus fees) accumulates in a single pool. Upon market resolution, the entire pool is distributed among winners proportionally to their share of the winning outcome pool.

2.1.1 Probability Derivation

At any point in time, the implied probability of each outcome is derived from the ETH allocation:

```
P(YES) = yesPool / (yesPool + noPool)
P(NO)  = noPool  / (yesPool + noPool)
```

This is analogous to real-world horse racing parimutuel systems, where odds shift dynamically as more bets are placed.

2.1.2 Payout Calculation

When a market resolves, winning traders receive a proportional share of the total pool:

```
payout(user) = (userShares / winningPool) × totalPool
```

Where userShares is the ETH committed by the user to the winning outcome, and winningPool is the total ETH committed by all traders to the winning outcome.

Example: If totalPool = 1 ETH, winningPool = 0.3 ETH, and userShares = 0.1 ETH:

```
payout = (0.1 / 0.3) × 1 = 0.333 ETH (3.33x return)
```

2.2 Fee Structure

A protocol fee is applied at the point of share purchase, not at claim time. This ensures the fee is predictable and transparent at trade time.

Parameter	Value	Notes
Protocol Fee	2% (200 bps)	Applied to each trade
Max Fee	5% (500 bps)	Hard cap in contract
Fee Recipient	Factory deployer	Configurable by owner
Fee Collection	At trade time	From msg.value before pool deposit

2.3 Market Lifecycle

Each market passes through the following states:

State	Description
OPEN	Trading is active. Users can buy shares.
EXPIRED	expirationTime passed. No new trades allowed.
RESOLVED	Oracle confirmed outcome. Claims enabled.
INVALID	Oracle returned INVALID. All shares refunded proportionally.

3. Smart Contract Architecture

3.1 Core Contracts

Contract	Responsibility
MarketFactory.sol	Deploys PredictionMarket instances. Manages global oracle, fees, and Phase 3 integrations.
PredictionMarket.sol	Individual YES/NO parimutuel market. Holds pool ETH, tracks shares, handles resolution and claims.
MultiMarketFactory.sol	Deploys MultiOutcomeMarket instances. Mirrors MarketFactory for multi-outcome markets.
MultiOutcomeMarket.sol	Supports 2–10 outcome parimutuel market. Separate pool per outcome.
MultiOracle.sol	Stores winning outcome index for multi-outcome markets. Permissioned resolver.

3.2 Phase 3 Contracts

Contract	Responsibility
PREDToken.sol	ERC20 governance token. 100M max supply. Minter role assigned to LiquidityMining.
LiquidityMining.sol	Distributes PRED rewards. 100 PRED per market created, 10 PRED per trade. Authorized callers only.
ReferralSystem.sol	On-chain referral tracking. Stores referrer mapping. 0.5% ETH fee per referred trade.
ChainlinkOracle.sol	Reads Chainlink price feeds on Sepolia. Provides live ETH/USD and BTC/USD prices.

3.3 Contract Interactions

The following sequence describes a complete market lifecycle:

- User calls MarketFactory.createMarket(question, category, expirationTime)
- Factory deploys new PredictionMarket with gas override of 3,000,000
- Factory calls LiquidityMining.recordCreation(creator) — awards 100 PRED pending
- Users call PredictionMarket.buyYesShares() or buyNoShares() with ETH
- Each trade calls LiquidityMining.recordTrade(trader) — awards 10 PRED pending
- After expirationTime passes, any address calls PredictionMarket.resolve()
- resolve() queries the oracle, stores outcome, enables claim()
- Winners call claimReward() to receive proportional ETH payout

4. Oracle Design

4.1 MockOracle (Testing)

During development, a MockOracle contract was used to manually set market outcomes for testing. The factory owner calls setResolution(marketId, outcome) to resolve markets.

4.2 ChainlinkOracle (Production)

For price-conditional markets, the ChainlinkOracle reads from Chainlink AggregatorV3Interface contracts on Sepolia:

Feed	Address	Decimals
ETH/USD	0x694AA1769357215DE4FAC081bf1f309aDC325306	8
BTC/USD	0x1b44F3514812d835EB1BDB0acB33d3fA3351Ee43	8
LINK/USD	0xc59E3633BAAC79493d908e63626716e204A45EdF	8

Market owners configure target prices and direction (resolve if price > target or < target). Anyone can call tryResolve() once the condition is met.

5. Incentive System

5.1 PRED Token

PRED is the native utility token of the PredictX protocol. It is non-transferable at the protocol level and serves as a participation reward.

Property	Value
Token Name	PredictX
Symbol	PRED
Max Supply	100,000,000 PRED
Initial Mint	10,000,000 PRED (to deployer)
Minting Authority	LiquidityMining contract only
Standard	ERC20 (OpenZeppelin)

5.2 Liquidity Mining Rewards

Action	PRED Reward	Trigger
Create Market	100 PRED	Automatic via factory
Place Trade	10 PRED	Automatic via market contract
Claim PRED	—	Manual call to LiquidityMining.claimRewards()

5.3 Referral System

The on-chain referral system incentivizes user acquisition. Referrers earn ETH (not PRED) from each trade their referrals make.

```
referralFee = tradeAmount × referralFeeBps / 10000

referralFeeBps = 50 (0.5%)
```

Referral earnings accumulate in the ReferralSystem contract and can be claimed at any time via claimEarnings().

6. The Graph Integration

PredictX uses a custom subgraph deployed to The Graph Studio to efficiently index market events. Direct RPC reads are expensive and slow for listing all markets — The Graph solves this.

6.1 Indexed Events

Event	Source	Data Indexed
MarketCreated	MarketFactory	marketId, address, creator, question, expirationTime
SharesPurchased	PredictionMarket	buyer, isYes, amount, shares, timestamp
MarketResolved	PredictionMarket	outcome, totalPool, timestamp

6.2 Category Indexing

The MarketCreated event does not include the category field (to save gas). Instead, the subgraph mapping calls PredictionMarket.getMarketInfo() at index time via a contract call to retrieve the category, enabling category-based filtering in the frontend.

7. Frontend Architecture

7.1 Technology Choices

Technology	Choice	Reason
Framework	Next.js 14 App Router	SSR, route-based code splitting, Vercel deployment
Web3 Library	wagmi v2 + viem	Type-safe, React hooks, multicall support
Wallet UI	RainbowKit	Multi-wallet support, mobile friendly
Styling	Inline styles only	Avoids Tailwind/dark mode class conflicts in SSR
State Management	React Query via wagmi	Automatic refetching, caching, loading states
Indexing	The Graph (GraphQL)	Fast market listing, trade history, creator stats

7.2 SSR Considerations

The wagmi configuration disables SSR (ssr: false) to prevent wallet disconnects on navigation. All wallet state is deferred to client-side rendering using useState(null) mounting guards.

8. Security Considerations

See the accompanying Security Paper for a full threat model and analysis. Key design decisions with security implications:

- ReentrancyGuard on all ETH-transferring functions (buyShares, claimReward, resolve)
- Checks-Effects-Interactions pattern in all state-changing functions
- Oracle-dependent resolution — markets cannot self-resolve without external confirmation
- Factory-controlled child contract deployment — no arbitrary contract deployment
- Fee recipient is immutable per-market — set at deployment, not changeable after
- Max fee hard cap (5%) enforced in constructor — prevents fee manipulation

9. Known Limitations

PredictX is currently a testnet deployment and has the following known limitations:

- No formal audit has been performed
- Oracle resolution is manual (admin-controlled) for most markets
- The PRED token has no governance utility yet
- Multi-outcome markets require manual oracle resolution via admin
- Gas costs for market creation (~3M gas) may be prohibitive on mainnet
- No time-weighted price oracle — susceptible to last-block manipulation at expiry

10. Roadmap

Phase	Features
Phase 1 (Complete)	YES/NO markets, trading UI, admin panel, leaderboard
Phase 2 (Complete)	The Graph subgraph, trade history, portfolio dashboard
Phase 3 (Complete)	PRED token, liquidity mining, referral system, Chainlink oracle
Phase 4 (Complete)	Multi-outcome markets (2–10 outcomes)
Phase 5 (Planned)	Formal audit, mainnet/Base L2 deployment
Phase 6 (Planned)	PRED governance, DAO treasury, fee distribution to stakers
Phase 7 (Planned)	Automated oracle via Chainlink Any API or UMA
Phase 8 (Planned)	Mobile app, push notifications

11. References

Arrow, K. J., Forsythe, R., Gorham, M., Hahn, R., Hanson, R., Ledyard, J. O., ... & Zitzewitz, E. (2008). Promise of prediction markets. Science, 320(5878), 877-878.

Sunstein, C. R. (2006). Infotopia: How many minds produce knowledge. Oxford University Press.

Buterin, V. (2013). Ethereum Whitepaper. ethereum.org.

Chainlink Labs. (2021). Chainlink 2.0: Next Steps in the Evolution of Decentralized Oracle Networks.

The Graph Protocol. (2020). The Graph: A Decentralized Query Protocol for Blockchains.

Appendix A — Deployed Contract Addresses

Contract	Address (Sepolia)
MarketFactory	0x51430273cA467Fd6a961598B5bcD28d6532A8D33
ChainlinkOracle	0x4cb12c69E85A280C41815805C1446b121E8c5462
PREDToken	0x1a5ecdbCbe1931C4e745B82B3C8E09CBc4015C49
LiquidityMining	0xAC8e774dd8218D716F455AB7872E7c0843985981
ReferralSystem	0xaBa4F2D457CE0fEf0C06A1e89A3662980C8e1F4A
MultiOracle	0x1aB76B758Cb2c45Ca6E876294F7972133Ebd1619
MultiMarketFactory	0x30a99B8A1C7b71314160c0396b49eE9db8bbC4Ab

Appendix B — The Graph Subgraph

Property	Value
Subgraph Name	predict-x
Version	v0.0.4
Network	Sepolia
Studio ID	1744854
Endpoint	https://api.studio.thegraph.com/query/1744854/predict-x/v0.0.4